

sort() 콜백 — JavaScript 정렬의 핵심

JS의 sort()는 콜백 없이 쓰면 문자열 기준 정렬이라는 함정이 있다. 코딩테스트에서 반드시 알아야 할 핵심.

위치: `Array.prototype.sort()` — 콜백 함수가 Java의 Comparator 역할

sort() 기본 동작 — 함정 (치명적)

```
// 숫자를 정렬하면 문자열 기준으로 정렬된다!
const nums = [10, 9, 1, 2];
nums.sort();
console.log(nums); // [1, 10, 2, 9] - 사전순이라 10이 2보다 앞!

// 반드시 콜백을 넘겨야 숫자 정렬이 올바르게 동작한다
const correct = [10, 9, 1, 2];
correct.sort((a, b) => a - b);
console.log(correct); // [1, 2, 9, 10]
```

이것을 모르면 코딩테스트에서 틀린다. 숫자 정렬 시 반드시 `(a, b) => a - b` 사용.

핵심 콜백 패턴

콜백	정렬 방식
<code>(a, b) => a - b</code>	숫자 오름차순
<code>(a, b) => b - a</code>	숫자 내림차순
<code>(a, b) => a.localeCompare(b)</code>	문자열 사전순 (유니코드 올바르게 처리)
<code>(a, b) => b.localeCompare(a)</code>	문자열 역사전순

콜백 반환값의 의미:

- 음수 → a가 b보다 앞
- 0 → 순서 유지
- 양수 → b가 a보다 앞

패턴

코딩테스트 출제 빈도: 패턴1 기본 오름차순/내림차순 > 패턴2 문자열 이어붙이기 커스텀 정렬 > 패턴3 내림차순 + 인덱스 조건 탐색 > 패턴4 다단계 정렬

패턴 1: 기본 오름차순/내림차순

언제 쓰는가

- 단순 정렬이 필요할 때

실무에서는

- 가격순 정렬, 날짜순 정렬

```
const prices = [3500, 1200, 5000, 800, 2200];

// 오름차순 (싼 것부터)
const asc = [...prices].sort((a, b) => a - b);
console.log(asc); // [800, 1200, 2200, 3500, 5000]

// 내림차순 (비싼 것부터)
const desc = [...prices].sort((a, b) => b - a);
console.log(desc); // [5000, 3500, 2200, 1200, 800]

// 문자열 정렬
const names = ["Charlie", "Alice", "Bob"];
names.sort((a, b) => a.localeCompare(b));
console.log(names); // ["Alice", "Bob", "Charlie"]
```

패턴 2: 문자열 이어붙이기 커스텀 정렬 — "가장 큰 수" 패턴

언제 쓰는가

- 기본 정렬 기준으로 해결할 수 없는 특수한 정렬
- 대표 문제: 숫자 배열을 이어붙여서 가장 큰 수 만들기

실무에서는

- 버전 번호 정렬

```
// 숫자를 이어붙여 가장 큰 수 만들기: [6, 10, 2] → "6210"
function largestNumber(nums) {
  const sorted = nums
    .map(String) // 문자열로 변환
    .sort((a, b) => (b + a) - (a + b)); // "610" vs "106" 비교

  // 모두 0인 경우 처리
  if (sorted[0] === "0") return "0";
  return sorted.join("");
}

console.log(largestNumber([6, 10, 2])); // "6210"
console.log(largestNumber([3, 30, 34, 5, 9])); // "9534330"
console.log(largestNumber([0, 0, 0])); // "0"
```

패턴 3: 내림차순 + 인덱스 조건 탐색

언제 쓰는가

- 가장 큰 값부터 조건을 확인해야 할 때

실무에서는

- Top-N 추출

```
// 로프 문제: 내림차순 정렬 후 i번째 값 * (i+1)의 최대값
function maxWeight(ropes) {
  ropes.sort((a, b) => b - a); // 내림차순

  let max = 0;
  for (let i = 0; i < ropes.length; i++) {
    // i번째 로프까지 사용 → 가장 약한 로프 * 개수
    const weight = ropes[i] * (i + 1);
    max = Math.max(max, weight);
  }
  return max;
}

console.log(maxWeight([2, 10, 8])); // 16 (8 * 2)
console.log(maxWeight([1, 5, 3, 7])); // 12 (3 * 4 또는 5 * 2... → 7*1=7,
5*2=10, 3*3=9, 1*4=4 → 10)
// 실제: [7,5,3,1] → 7*1=7, 5*2=10, 3*3=9, 1*4=4 → 최대 10
```

패턴 4: 다단계 정렬 — 콜백 내 다중 조건

언제 쓰는가

- 1차 기준이 같을 때 2차 기준으로 정렬해야 할 때

실무에서는

- 멀티 컬럼 테이블 정렬

```
const students = [
  { name: "Charlie", score: 85 },
  { name: "Alice", score: 92 },
  { name: "Bob", score: 85 },
  { name: "David", score: 92 },
];

// 점수 내림차순 → 같으면 이름 오름차순
students.sort((a, b) => {
  if (a.score !== b.score) return b.score - a.score; // 점수 내림차순
```

```

    return a.name.localeCompare(b.name); // 이름 오름차순
});

console.log(students);
// [
//   { name: "Alice", score: 92 },
//   { name: "David", score: 92 },
//   { name: "Bob", score: 85 },
//   { name: "Charlie", score: 85 }
// ]

// || 연산자로 더 간결하게 (0이면 다음 조건으로)
students.sort((a, b) => b.score - a.score ||
a.name.localeCompare(b.name));

```

주의사항

sort()는 원본을 변경한다 — 복사 후 정렬하기

```

const original = [3, 1, 2];
const sorted = [...original].sort((a, b) => a - b); // 스프레드로 복사
console.log(original); // [3, 1, 2] - 원본 보존
console.log(sorted);   // [1, 2, 3]

```

안정 정렬 (Stable Sort)

- ES2019부터 `Array.prototype.sort()`는 안정 정렬 보장
- 그 이전에는 브라우저/엔진에 따라 달랐음
- 안정 정렬: 같은 키 값을 가진 요소들의 원래 순서 유지

toSorted() — 원본 불변 정렬 (ES2023)

```

// ES2023 이상에서만 사용 가능
const arr = [3, 1, 2];
const sorted = arr.toSorted((a, b) => a - b);
console.log(arr); // [3, 1, 2] - 원본 보존
console.log(sorted); // [1, 2, 3]
// 코딩테스트 환경에 따라 지원되지 않을 수 있으므로 [...arr].sort() 권장

```